Задача 1.

Мини-интерпретатор λ-исчисления: представление термов, свободные переменные и подстановка

В этой задаче нужно представить термы λ-исчисления в виде структур данных Clojure и реализовать две базовые операции над ними. На этом этапе мы не выполняем β-редукцию и не решаем проблему захвата переменных при подстановке.

1. Представление λ-термов

Будем использовать векторы с ключевым словом в первой позиции. Ключевое слово определяет тип узла AST.

```
(defn variable [x]
 [:var x])

(defn lambda [x body]
 [:lambda x body])

(defn app [f x]
 [:app f x])
```

Предикаты для удобства:

```
(defn variable? [term]
 (= :var (first term)))


(defn lambda? [term]
 (= :lambda (first term)))


(defn app? [term]
 (= :app (first term)))
```

- [:var x] — переменная x.
- [:lambda x body] — λ-абстракция λx.body.
- [:app f x] — аппликация f к аргументу x.

2. Примеры построения термов

Переменная:

```
(variable 'x)
```

Соответствующая λ-нотация:

$$x$$

Тождественная функция:

```
(lambda 'x
 (variable 'x))
```

Соответствующая λ-нотация:

$$\lambda x.x$$

Аппликация тождественной функции к y:

```
(app
 (lambda 'x
  (variable 'x))
 (variable 'y))
```

Соответствующая λ-нотация:

$$(\lambda x.x)\ y$$

Терм с несколькими переменными:

```
(lambda 'x
 (app
  (app
   (variable 'x)
   (variable 'y))
  (variable 'z)))
```

Соответствующая λ-нотация:

$$\lambda x.((x\ y)\ z)$$

Обратите внимание: символы x, y, z хранятся как данные, поэтому при непосредственном создании терма используются quoted symbols: 'x, 'y, 'z.

1a. Свободные переменные — free-vars

Реализуйте функцию:

```
(defn free-vars [term]
 ;; TODO
 )
```

Функция должна возвращать множество (set) или список/вектор без дублирования свободных переменных терма.

Используйте следующие правила:

- FV(x) = {x}
- FV(M N) = FV(M) ∪ FV(N)
- FV(λx.M) = FV(M) \ {x}

Примеры:

```
(free-vars (variable 'x))
;; => #{x}

(free-vars
 (lambda 'x
  (variable 'x)))
;; => #{}

(free-vars
 (lambda 'x
  (app
   (variable 'x)
   (variable 'y))))
;; => #{y}
```

Важно: параметр λ-абстракции связывает только соответствующие вхождения переменной в теле этой абстракции. Остальные переменные остаются свободными.

1b. Наивная подстановка — naive-substitute

Реализуйте функцию:

```
(defn naive-substitute [term x replacement]
 ;; TODO
 )
```

Функция должна заменить свободные вхождения переменной x в term на терм replacement.

На этом этапе используем наивную подстановку:

- связанную переменную заменять нельзя;
- если параметр λ-абстракции совпадает с x, внутрь этой λ-абстракции подстановка не продолжается;
- защиту от захвата переменных (variable capture) реализовывать пока не нужно; α-переименование при подстановке не выполняется.

Примеры:

```
(naive-substitute
 (variable 'x)
 'x
 (variable 'y))
;; => [:var y]

(naive-substitute
 (lambda 'x
```

```
  (variable 'x))
 'x
 (variable 'y))
;; => [:lambda x [:var x]]

(naive-substitute
 (lambda 'y
  (variable 'x))
 'x
 (variable 'y))
;; => [:lambda y [:var y]]
```

Последний пример специально демонстрирует проблему наивной подстановки. До подстановки y в replacement была свободной переменной, а после подстановки оказалась под λy и стала связанной. Это называется захватом переменной. Исправление этой проблемы не требуется, но если уверены в своих силах, то попробуйте.

Требования к решению

- Не использовать eval для обработки термов.
- Разбирать структуру терма рекурсивно.
- Использовать созданные предикаты variable?, lambda? и app? при разборе термов.
- free-vars должна возвращать Clojure set или список/вектор без повторов.
- naive-substitute должна корректно учитывать область связывания λ-параметра, но может допускать variable capture.

Подсказка: удобно сначала определить тип term, а затем рекурсивно обработать его в зависимости от того, является ли он переменной, λ-абстракцией или аппликацией.